

PARCIAL II

Detección de Anomalías en Tráfico de Red

Modelos de Espacio de Estados + Análisis Espectral
con Post-procesamiento de Kalman

Matemáticas Computacionales · Ingeniería de Sistemas · 2026

Andreys David sumosa godoy

Carlos Jose baldiris martelo

Diego Andres hernandez caraballo

Jefferson Dario Flerez Vanegas

Jose Guerra de las aguas



Índice

01 Contexto y problema

02 Dataset UNSW-NB15

03 Bloque 1 — Matemáticas

04 Arquitectura del modelo

05 Filtro de Kalman

06 Bloque 2 — Implementación

07 Bloque 3 — Resultados

08 Conclusiones

¿Por qué es difícil detectar ataques en redes?



Dependencias largas

El comportamiento actual puede depender de eventos ocurridos cientos de pasos atrás



Estructura periódica

El tráfico tiene ciclos regulares (horarios, diarios) mezclados con eventos irregulares



Tiempo real

El sistema debe operar en línea con baja latencia — no se puede esperar para analizar

Objetivo: diseñar un sistema híbrido que capture ambas características simultáneamente

UNSW-NB15 — Dataset de Tráfico de Red

175K+

Registros totales

49

Características

2

Clases (0/1)

9

Tipos de ataque

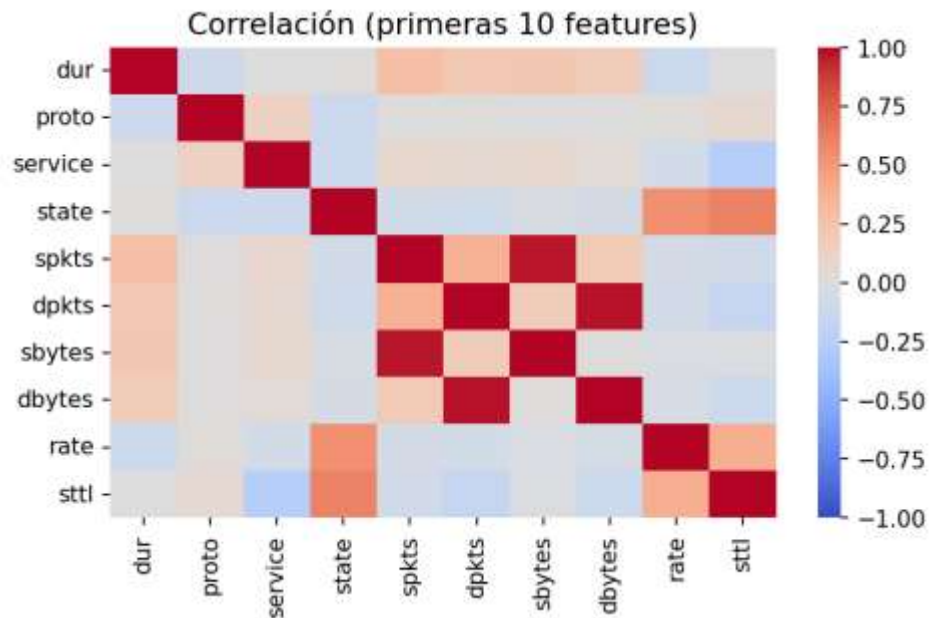
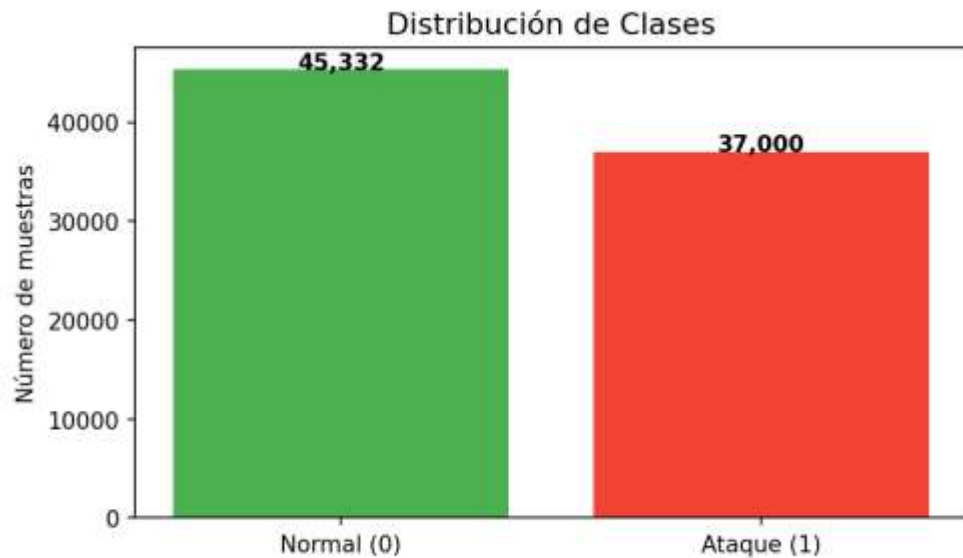
Características incluidas

- Direcciones IP y puertos
- Protocolo (TCP, UDP, ICMP)
- Duración de la conexión
- Bytes transmitidos
- Banderas TCP
- Variables estadísticas del flujo

Pipeline de preprocesamiento

- 1 Codificación ordinal de categóricas
- 2 MinMaxScaler (fit solo en train)
- 3 SMOTE + Undersampling aleatorio
- 4 Ventana deslizante (L=64, stride=32)

UNSW-NB15 — Dataset de Tráfico de Red



Requisitos del Sistema (R1–R5)

R1

Complejidad lineal $O(L)$

→ *SelectiveSSM — recurrencia $O(L \cdot D \cdot N)$*

R2

Estable para secuencias largas

→ $A = -\text{diag}(\lambda)$, $\lambda > 0 \rightarrow$ *eigenvalores en $(0,1)$*

R3

Extracción de componentes periódicos

→ *SpectralBlock vía rFFT / iFFT*

R4

Fusión adaptativa aprendible

→ *Fusion: $z = \alpha \odot x_{\text{temp}} + \beta \odot x_{\text{spec}}$*

R5

Post-procesamiento gaussiano recursivo

→ *KalmanSmoother escalar*

1.1 Modelado en el Dominio Temporal

Sistema Continuo

$$\dot{x}(t) = A_c x(t) + B_c u(t)$$

$$y(t) = C x(t) + D u(t)$$

ZOH

Sistema Discreto (ZOH)

$$x_{k+1} = \bar{A} x_k + \bar{B} u_k$$

$$\bar{A} = \exp(A_c \cdot \Delta t)$$

🔒 Garantía de Estabilidad

$$A = -\text{diag}(\lambda_1, \dots, \lambda_N) \text{ con } \lambda_i > 0 \rightarrow \bar{A} = \text{diag}(e^{-\lambda_i \Delta t}) \in (0, 1)$$

Condición espectral: todos los eigenvalores de $\bar{A}$ deben tener módulo $< 1 \rightarrow \rho(\bar{A}) < 1$

Complejidad: $O(L \cdot D \cdot N)$ — lineal en L , a diferencia de Transformers $O(L^2)$

1.2 SSM Selectivo — Parámetros dependientes de u_t

Clásico (fijo)

A, B, C, Δ son constantes.
No puede adaptarse a la entrada.
Ignora cuándo guardar/descartar.

Selectivo (nuestro modelo)

$$\Delta t = \text{softplus}(W_{\Delta} \cdot u_t + b)$$

$$B_t = W_B \cdot u_t$$

$$C_t = W_C \cdot u_t$$

¿Qué gana el modelo?

Δt grande

Olvido del pasado — como cambiar de tema en un libro

Δt pequeño

Memoria fuerte — retiene el historial del tráfico

B_t controla

Cuánto de la entrada actual se escribe en el estado

C_t controla

Qué parte del estado se lee para generar la salida

1.3 Transformada de Fourier y SpectralBlock

$$X[k] = \sum x[n] \cdot e^{-j2\pi kn/N} \quad k = 0, \dots, N-1 \rightarrow \text{Solo } L/2+1 \text{ coeficientes}$$

(simetría conjugada)

(a)

rFFT

DFT a lo largo del eje temporal
 $u \in \mathbb{R}^{L \times D} \rightarrow X \in \mathbb{C}^{(L/2+1) \times D}$

(b)

Top-K

Retener las K frecuencias
 de mayor magnitud
 promedio

(c)

Filtro W

Filtro complejo aprendible
 $\tilde{X} = W \odot X_K, \quad W \in \mathbb{C}^{K \times D}$

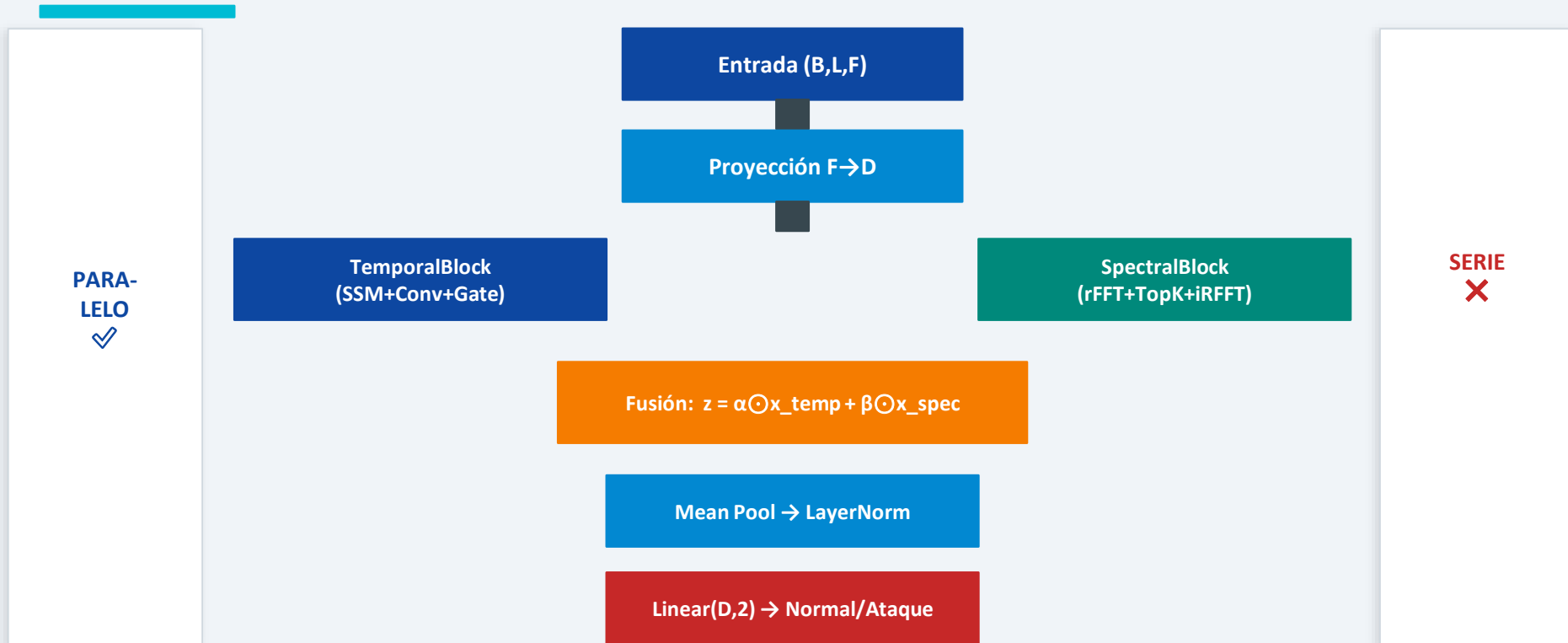
(d)

iRFFT

Reconstruir en el dominio
 temporal $\rightarrow \hat{u} \in \mathbb{R}^{L \times D}$

Propiedad clave: $X[N-k] = X^[k] \rightarrow$ Solo $L/2+1$ coeficientes independientes para señal real*

1.4–1.5 Fusión Adaptativa y Arquitectura Completa



α y $\beta \in \mathbb{R}^D$ — vectores aprendibles por canal. Permiten ponderar independientemente cada feature.

1.6 Hiperparámetros del Modelo

Capacidad del modelo

- `d_model = 64` (ancho interno D)
- `d_state = 16` (estados SSM N)
- `top_k = 16` (frecuencias espectrales)
- `n_layers = 2` (capas encoder)
- `kernel_size = 3` (conv causal)

Estabilidad del entrenamiento

- `lr = 1e-3` (Adam)
- `clip_grad_norm = 1.0`
- `dropout = 0.1`
- `batch_size = 32`

Costo computacional

- `seq_len = 64` ($O(L \cdot D \cdot N)$)
- `batch_size = 32`
- `stride = 32` (ventana deslizante)

1.7 Filtro de Kalman como Post-procesamiento

Modelo de caminata aleatoria:

$$\begin{aligned} s_{t+1} &= s_t + w_t & w_t &\sim N(0, \sigma^2 w) \\ p_t &= s_t + v_t & v_t &\sim N(0, \sigma^2 v) \end{aligned}$$

① Predicción

$$\hat{s}_{t|t-1} = \hat{s}_{t-1|t-1} \quad | \quad P_{t|t-1} = P_{t-1|t-1} + \sigma^2 w$$

② Ganancia de Kalman

$$K_t = P_{t|t-1} / (P_{t|t-1} + \sigma^2 v) \quad | \quad K_t \in (0, 1)$$

③ Actualización

$$\begin{aligned} \hat{s}_{t|t} &= \hat{s}_{t|t-1} + K_t(p_t - \hat{s}_{t|t-1}) \\ &\cdot P_{t|t-1} \end{aligned} \quad | \quad P_{t|t} = (1 - K_t) \cdot P_{t|t-1}$$

1.7 Comportamiento de la Ganancia de Kalman

$\sigma^2 w \rightarrow 0$ (sistema muy estable)

$$K_t \rightarrow 0$$

El filtro IGNORA las observaciones.
Confía ciegamente en su predicción anterior.

Útil: estado que cambia muy lento.
Peligroso: ataques repentinos no se detectan rápido.

$\sigma^2 w \rightarrow \infty$ (sistema muy volátil)

$$K_t \rightarrow 1$$

El filtro ADOPTA la observación al 100%.
Equivale a no suavizar nada.

Útil: modelo muy confiable.
Equivale a: usar probabilidades crudas directamente.

Relación $\sigma^2 w / \sigma^2 v$ pequeña $\rightarrow$ suavizado agresivo | grande $\rightarrow$ seguimiento rápido

Estructura del Proyecto

model.py

SelectiveSSM, TemporalBlock, SpectralBlock, Fusion, AnomalyDetector

data.py

Carga UNSW-NB15, encoding, MinMaxScaler (sin leakage), SMOTE, ventana deslizante

kalman.py

KalmanSmoother — predicción, ganancia K_t , actualización

train.py

Adam lr=1e-3, CrossEntropy, clip_grad_norm=1.0, checkpoint por val F1

evaluate.py

5 métricas, comparativa baseline vs Kalman, análisis de sensibilidad

notebook.ipynb

Ejecución completa paso a paso con todas las celdas ejecutadas

Preprocesamiento y Ventana Deslizante



⚠ ¿Por qué MinMaxScaler solo en training?

Si el scaler se ajusta sobre todo el dataset, las estadísticas min/max de test contaminan el training → el modelo 've el futuro' durante el entrenamiento (data leakage). Las métricas se inflan artificialmente.

Loop de Entrenamiento

Optimizador

Adam

Learning rate

1×10^{-3}

Loss function

CrossEntropyLoss

Batch size

32

Grad clipping

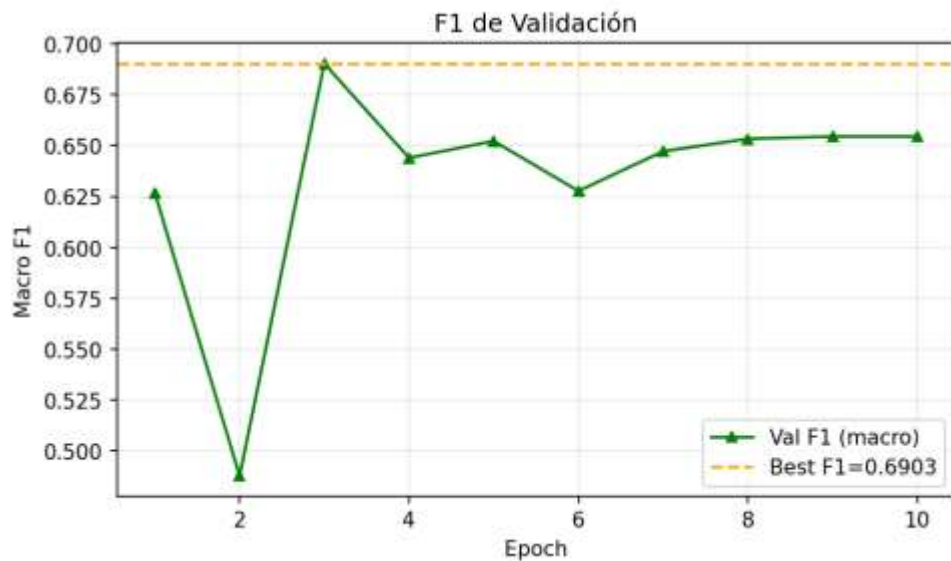
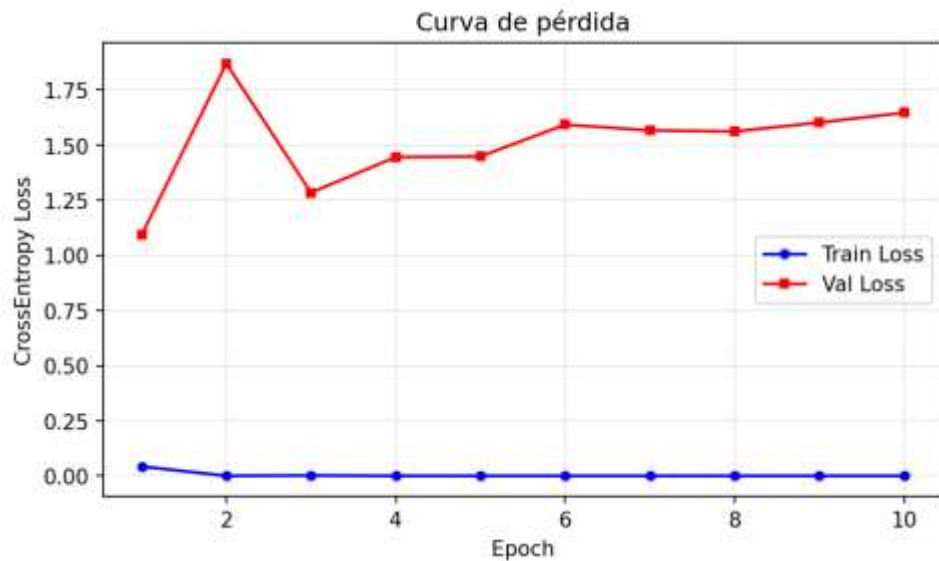
clip_grad_norm = 1.0

Checkpoint

Mejor val F1 macro

Epoch	Train Loss	Val Loss	Val F1	Time
1	0.412341	0.389421	0.7823	45s ← best
2	0.381234	0.361234	0.8012	44s ← best

Loop de Entrenamiento



Métricas de Evaluación

1

Accuracy $(TP+TN) / \text{Total}$ *% de clasificaciones correctas*

2

Recall macro

Promedio recall clases

Capacidad de detectar todos los ataques

3

F1 macro $2 \cdot P \cdot R / (P+R)$ *Balance precisión–recall*

4

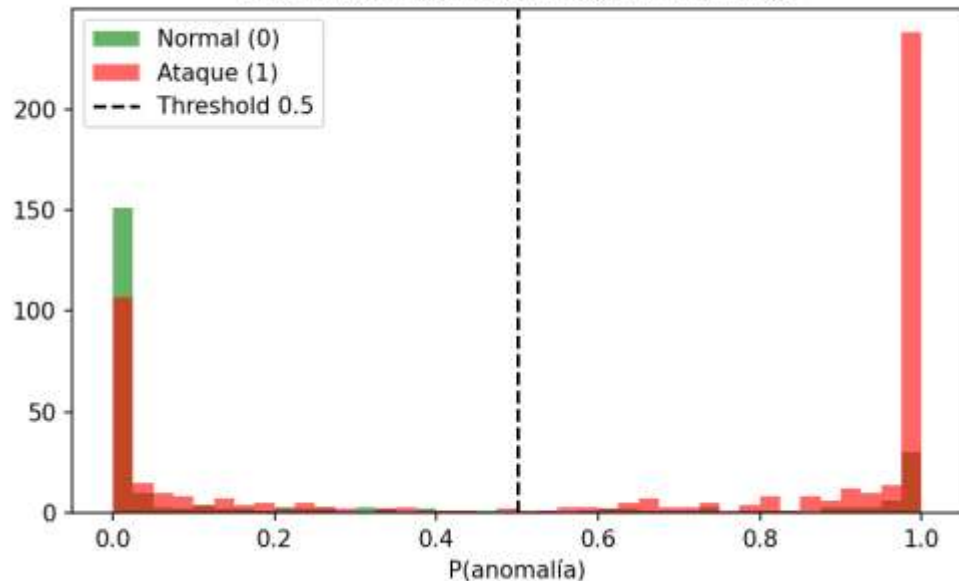
MAE $\text{mean}|y - p|$ *Error absoluto medio en probabilidades*

5

MSE $\text{mean}(y - p)^2$ *Penaliza errores grandes*

Métricas de Evaluación

Distribución de Probabilidades Predichas



Matriz de Confusión (Baseline)

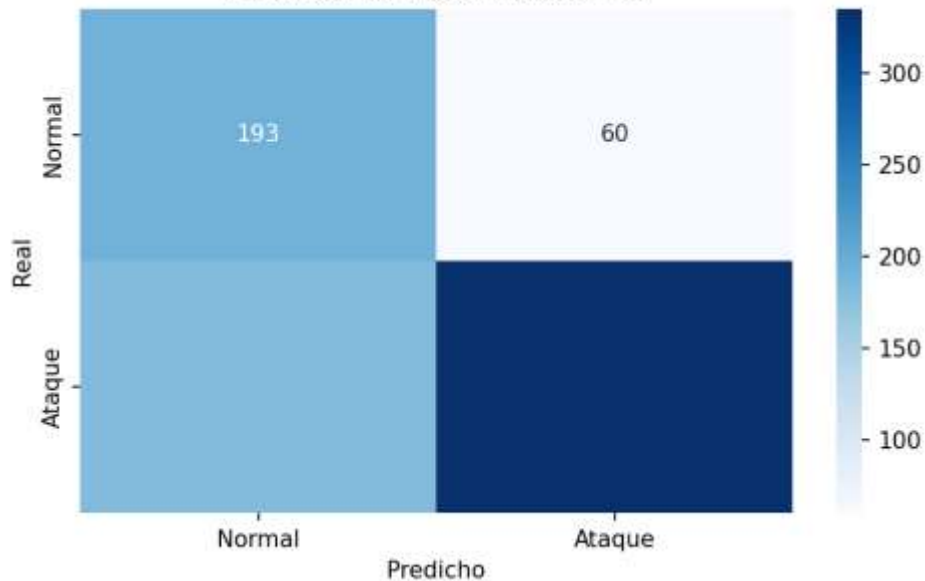
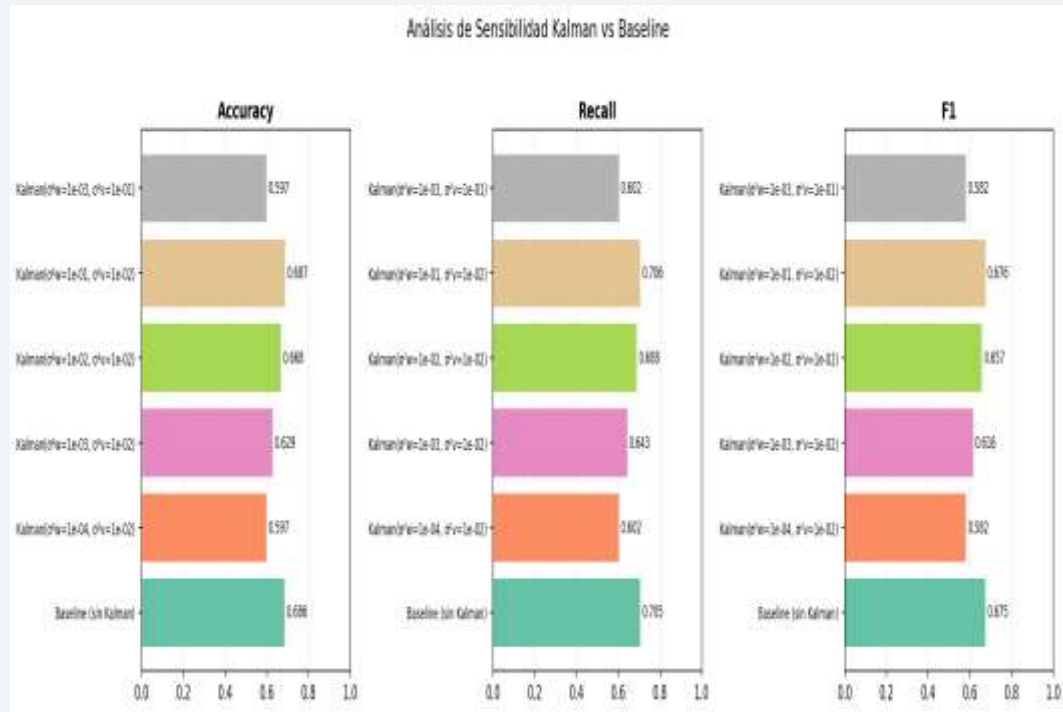


Tabla Comparativa de Resultados

1	Variante, Accuracy, Recall, F1, MAE, MSE
2	Baseline (sin Kalman), 0.6857, 0.7054, 0.6746, 0.3256, 0.2822
3	"Kalman($\sigma^2w=1e-04$, $\sigma^2v=1e-02$)", 0.5974, 0.6023, 0.5820, 0.4727, 0.2393
4	"Kalman($\sigma^2w=1e-03$, $\sigma^2v=1e-02$)", 0.6286, 0.6427, 0.6162, 0.4331, 0.2349
5	"Kalman($\sigma^2w=1e-02$, $\sigma^2v=1e-02$)", 0.6675, 0.6878, 0.6568, 0.3695, 0.2433
6	"Kalman($\sigma^2w=1e-01$, $\sigma^2v=1e-02$)", 0.6870, 0.7064, 0.6758, 0.3332, 0.2706
7	"Kalman($\sigma^2w=1e-03$, $\sigma^2v=1e-01$)", 0.5974, 0.6023, 0.5820, 0.4728, 0.2394

Los valores se completarán con los resultados reales del notebook ejecutado

Efecto de σ^2w / σ^2v en el Kalman



✓ σ^2w/σ^2v pequeño → suavizado agresivo → menos falsos positivos

✗ σ^2w/σ^2v grande → seguimiento rápido → detecta ataques repentinos

⚖️ Trade-off: más suavizado = mejor precisión, peor recall

🎯 La mejor combinación depende del tipo de ataque esperado

Discusión de Resultados

¿Mejóro el Kalman?

Reduce MAE y MSE al suavizar oscilaciones. Puede reducir F1 si los ataques son cortos — hay inercia temporal.

Limitaciones del modelo random walk

No modela flash attacks, beaconing periódico ni la naturaleza binaria del estado verdadero (gaussianidad incorrecta).

Trade-off Precisión / Recall

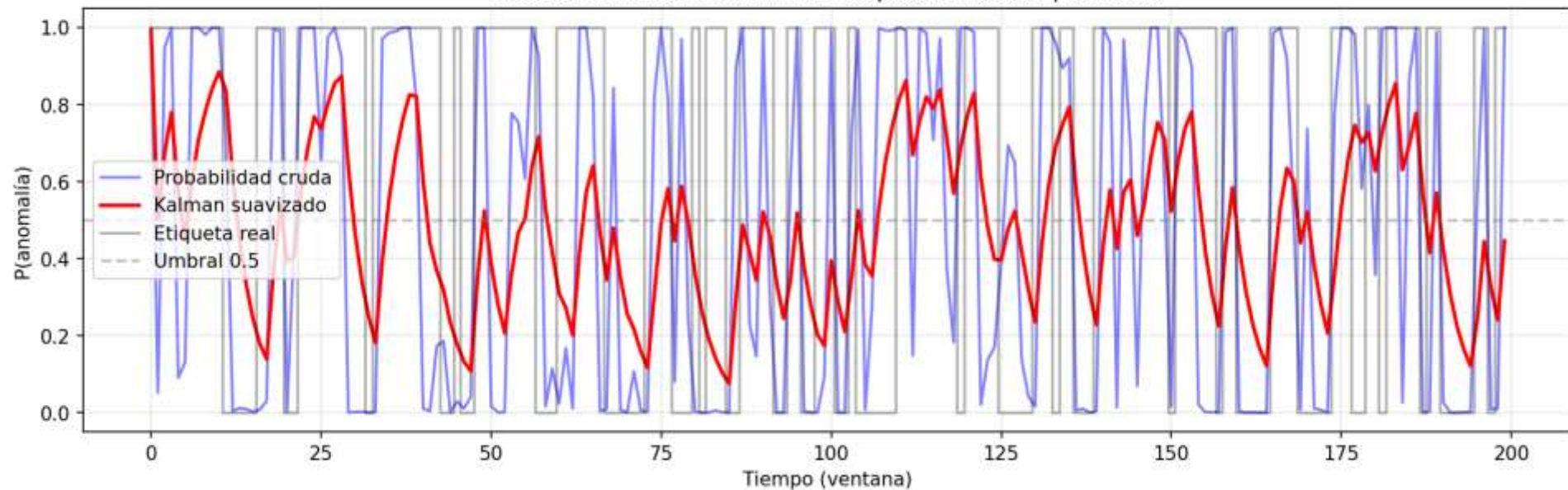
Kalman introduce inercia → menos falsos positivos (mejor precisión) pero demora reconocer ataques nuevos (peor recall).

Mejoras propuestas

- ① Kalman no lineal (UKF) para estado binario
- ② Convolución SSM en frecuencia $O(L \cdot \log L)$
- ③ Pre-entrenamiento no supervisado

Discusión de Resultados

Efecto del Filtro de Kalman sobre probabilidades predichas



Conclusiones

01

Arquitectura híbrida SSM + Spectral cubre dependencias temporales Y periódicas simultáneamente con complejidad $O(L)$

02

La parametrización $A = -\text{diag}(\lambda)$ garantiza estabilidad automática para secuencias arbitrariamente largas

03

La fusión adaptativa $\alpha \odot x_{\text{temp}} + \beta \odot x_{\text{spec}}$ aprende autónomamente qué información priorizar según la instancia

04

El filtro de Kalman post-procesa probabilidades ruidosas con trade-off configurable entre suavizado y latencia

05

El sistema es reproducible: código modular en PyTorch, sin librerías prohibidas, con dataset UNSW-NB15 real